Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Bu derste algoritmaları akış diyagramı yerine C koduyla rahatça yazabilmek için gereken programlama konuları tamamlanır: dizilerin ve matrislerin fonksiyonlara aktarılması, değerle çağırmanın iki sorunu, pointer'lar, referansla çağırma ve pointer aritmetiği. Dersin sonunda bir dizinin sıralı olup olmadığını kontrol etme ve bir dizinin frekanslarını (histogramını) bulma algoritmaları konuşulur.
- Dizilerin fonksiyonlara aktarılması (tekrar)
  - Dizi, adıyla gönderildiğinde referansla aktarılır: fonksiyonda yapılan değişiklik orijinal diziye yansır. Dizinin tek bir elemanı ise değerle aktarılır.
  - Fonksiyon tanımında dizi `int dizi[]` (boyut yazılmadan) ve boyutu ayrı bir `int size` parametresiyle alınır.
  - `dizi`, `&dizi[0]` ve `&dizi` aynı adresi yazdırır.
- Ders kitabının "survey" örneği (ortalama, medyan, mod)
  - Ortalama: dizinin sıralı olması gerekmez. Bir döngüyle toplanır ve eleman sayısına bölünür. n işlem.
  - Medyan: dizi sıralı olmalıdır. Önce sıralanır (bubble sort ile n^2 maliyet), sonra `size / 2` indisli eleman alınır. `medyan` fonksiyonu diziyi sıralama fonksiyonuna gönderir ve dizi referansla gittiği için orijinal dizi sıralanmış olur.
  - Mod: en sık geçen elemandır. Her elemanın kaç kez geçtiği ayrı bir frekans dizisinde sayılır. En büyük frekansa sahip eleman moddur.
  - Fonksiyonların her biri ayrı yazılmıştır. Yazdırma, sıralamanın içine konmamıştır.
  - Histogram görüntü işlemede de kullanılır: 0-255 arasındaki piksel değerlerinin çoğu 0'a yakınsa görüntü karanlık, 255'e yakınsa çok aydınlıktır.
  - Örnekteki bubble sort'un iç döngüsü `size - 1`'e kadar gider. Hoca bunu son elemana tekrar bakmamakla açıkladı; doğrusu sabit `size - 1` sınırı yalnız `a[j + 1]` erişiminin dizinin dışına çıkmasını önler. Sıralanmış son elemanları atlamak için `size - pass` gibi bir sınır gerekir. Bir turda hiç yer değiştirme olmazsa durmak için bir bayrak eklenebilir.
- `#define` hakkında bir öğrenci sorusu
  - `#define SIZE 99` bir yerine koymadır (substitution). Derleme öncesinde `SIZE` yazılan her yere 99 kopyalanır. Tip kontrolü yapılmaz. `SIZE` yerine anlamsız bir şey tanımlanırsa hata ancak kopyalandığı yerde ortaya çıkar.
- İkili aramanın `return` kullanmadan yazılması
  - Kitaptaki örnekte döngünün içinde bulununca `return` yapılıyordu. Hoca, "aranan değer `b[middle]`'dan farklı" şartını `while` şartına taşımayı gösterir. Bunun için `middle` döngüden önce hesaplanmalı ya da `do-while` kullanılmalıdır.
- Matrislerin (iki boyutlu dizilerin) fonksiyonlara aktarılması
  - Parametrede sütun sayısı yazılmak zorundadır: `void printArray(int a[][3])`. Satır sayısı yazılmaz. Sütun sayısı yazılmazsa derleyici hata verir.
  - Satır sayısı ayrı bir parametreyle gönderilebilir: `void printArray(int a[][3], int satir)`. Böylece fonksiyon farklı satır sayılarında çalışır.
  - Kitaptaki öğrenci notları örneğinde `#define STUDENTS 3` ve `#define EXAMS 4` ile tanımlanan matris, `minimum`, `maximum` ve `average` fonksiyonlarına gönderilir. Satır ve sütun sayıları değerle aktarılır.
- Değerle çağırmanın iki sorunu
  - Birinci sorun: fonksiyon içinde orijinal değişkenler değiştirilemez. Örneğin `swap(x,`

`y)` sonrasında `x` ve `y` aynı kalır.
  - İkinci sorun: bir fonksiyon `return` ile en fazla tek bir değer döndürebilir. `swap`'ı `int` döndürecek şekilde yazsak bile iki değerden yalnızca biri geri alınabilir.
  - Çözüm referansla çağırmadır. C'de bu, pointer'larla dolaylı olarak yapılır. Bu gerçek bir referansla çağırma değildir, onu "simüle eder". Pointer bir algoritma konusu değil, programlama konusudur.
  - `scanf("%d", &a)` referansla, `printf("%d", a)` değerle çağırmadır. `printf` değeri değiştirmez, `scanf` değişkenin adresine yazar.
- Pointer'lar
  - Pointer, bir değişkenin bellekteki adresini tutan değişkendir. Tanımında değişken adının önüne * konur: `int *px;`. `px`'in tipi `int *`'dır (int pointer). `int` ile `int *` farklı tiplerdir. `px = x;` veya `x = px;` yazılamaz.
  - Her tipin kendi pointer'ı vardır: `char *pc;`, `double *pd;` vb.
  - Adres operatörü `&` (referencing): `px = &x;` ile `x`'in adresi `px`'e kopyalanır. Artık `px`, `x`'i gösterir.
  - İçerik (dereference, indirection) operatörü *: `*px`, `px`'in gösterdiği yerdeki değerdir. `x++` ile `(*px)++` aynı işi yapar.
  - Pointer'a ilk değer olarak 0, `NULL` veya bir adres verilebilir.
  - Örnek (`y = 5`, `yPtr = &y`): `y` 600000 adresinde olsun. `yPtr`'nin de kendi adresi vardır, ama içinde 600000 saklanır. `*yPtr = 10` ile `y` 10 olur.
  - * ve `&` birbirinin tersidir ve birbirini götürür: `*&y` ile `y`, `&*yPtr` ile `yPtr` aynıdır.
  - Kitap örneğinde `a = 7`, `aPtr = &a` için `&a` ve `aPtr` aynı adresi, `a` ve `*aPtr` aynı değeri (7) yazdırır. Adresler her çalıştırmada farklı olabilir.
- Pointer'larla referansla çağırma
  - Fonksiyon parametresi pointer olarak tanımlanır, çağrıda değişkenin adresi gönderilir: `void cubeByReference(int *nPtr) { *nPtr = *nPtr * *nPtr * *nPtr; }`, çağrısı `cubeByReference(&number);`. `number` 5'ten 125'e değişir.
  - Karşılaştırma: `cubeByValue(number)` 125'i `return` ile döndürür ve `number = cubeByValue(number);` atamasıyla değişir. Fonksiyon bitince içindeki yerel kopya yok olur.
  - Pointer ile gönderirken de aslında değer kopyalanır: kopyalanan değer, değişkenin adresidir. Hoca bu yüzden buna "yalancı call by reference" der.
  - Bu yöntemle bir fonksiyon birden fazla değişkeni değiştirebilir. Bu da birden fazla değer "döndürmek" demektir.
  - Bir fonksiyonda dizi parametresi `int a[]` yerine `int *a` olarak da yazılabilir. Dizinin adı zaten bir adrestir. Kitaptaki bubble sort örneği diziyi bu şekilde alır.
  - Kitaptaki `convertToUppercase(char *sPtr)` örneği, string'i karakter pointer'ı olarak alır. `*sPtr != '\0'` olduğu sürece karakter küçük harfse (`islower`) büyük harfe çevrilir (`toupper`) ve `sPtr++` ile sonraki karaktere geçilir.
  - `const` niteleyicisi ve dinamik bellek bu dersin konusu değildir.
- Derste yazılan örnekler
  - `swap_value(double x, double y)` çağrısından sonra `x = 5`, `y = 6` aynı kaldı. `swap_ref(double *x, double *y)` ile `t = *x; *x = *y; *y = t;` yapılınca değerler 6 ve 5 oldu.
  - `print_array(int *a, int size)` ve `mutate_array(int *a, int size)`: `mutate_array` içinde `a[3] = 444` yapılınca değişiklik `main`'deki diziye yansıdı. Aynı fonksiyonda `size--` yapılsa bile `main`'deki boyut değişmez, çünkü `size` değerle aktarılır. Her parametrenin aktarım şekli kendine özgüdür.
  - Prototipler yazılmadığında Dev-C++ "conflicting types" uyarıları verdi. Fonksiyonlar `main`'den önce tanımlanırsa prototipe gerek kalmaz. Prototipi olan ama tanımı olmayan bir fonksiyon çağrılırsa bağlama aşamasında "undefined reference" hatası alınır.
- Pointer aritmetiği
  - `int A[5] = {1, 2, 3};` dizisinde kalan iki eleman otomatik olarak 0 olur. `pa = A;` ile `pa` dizinin ilk elemanını gösterir. `pa = &A[0];` de aynı adresi verir. `pa = A[0];` yazılamaz: sol tarafta pointer, sağ tarafta `int` vardır.
  - `pa++` (veya `pa = pa + 1`) adresi 1 değil, gösterilen tipin boyutu kadar artırır. `int` 4 byte olduğu için adres 4 artar ve `pa` bir sonraki elemanı gösterir. `char` için 1, 8 byte'lık bir tip için 8 artar.
  - Dizinin adı (`A`) her zaman dizinin başlangıç adresini tutar ve değiştirilemez. `A++` yazılamaz. Pointer aritmetiği `pa` gibi ayrı bir pointer değişkeniyle yapılır.

  - `(*pa)++`: `pa`'nın gösterdiği değeri bir artırır. Örneğin 2 olan `A[1]` 3 olur, adres değişmez.
  - Diziyi yazdırmanın eşdeğer yolları: `a[i]`, `*(a + i)`. Fonksiyona gelen pointer'la `*a` yazdırıp `a++` ile ilerlemek de olur. Bu durumda fonksiyondaki yerel `a` değişir, ama `main`'deki dizi adı değişmez. `*(a + i)` yazılırsa `a` hiç değişmez. Parantez kullanmak operatör önceliğinden kaynaklanan karışıklığı önler.
  - Pointer'lar karşılaştırılabilir (hangisinin daha yüksek adreste olduğu) ve aynı tipteki pointer'lar birbirine atanabilir. Tek istisna `void *`'dır. Her tip pointer `void *`'a atanabilir ve gerekince tip dönüşümüyle geri alınır. Bu gelecek dönemin konusudur.
  - `bPtr = b;` ise `*(bPtr + 3)` dizinin 4. elemanıdır. `b[3]`, `*(b + 3)`, `bPtr[3]` ve `*(bPtr + 3)` aynı sonucu verir. Derleyici `b[i]` yazımını arka planda `*(b + i)`'ye çevirir.
  - Pointer dizileri ve fonksiyon pointer'ları gelecek dönemin konusudur.
- Dizinin sıralı olup olmadığını kontrol etme
  - Öğrencilerle birlikte kurulan çözüm: `i = 0`'dan başla, `while (i < n - 1 && a[i] < a[i+1]) i++;`. Döngüden sonra `i == n - 1` ise dizi sıralıdır.
  - `i` 1'den başlatılırsa `a[i-1]` ile `a[i]` karşılaştırılır ve döngü `i < n` olduğu sürece döner. Sonda `i == n` ise sıralıdır.
  - `&&`'nin sol tarafı yanlışsa sağ tarafına bakılmaz. Bu yüzden sınır kontrolü (`i < n - 1`) önce yazılmalıdır. Aksi hâlde dizinin dışına erişilebilir.
  - Daha kötü bir yöntem: diziyi sıralayıp sonucu karşılaştırmak. Sıralama en iyi ihtimalle $n \log n$, bubble sort ile n^2 işlem ister. Doğrudan kontrol ise n işlemdir.
- Frekans (histogram) hesabı
  - Soru: 0-9 arasındaki sayılardan oluşan bir dizide her sayıdan kaç tane var?
  - 1. yöntem: dış döngü dizinin elemanlarını, iç döngü 0'dan 9'a olası değerleri gezer. `if (A[i] == j) B[j]++;`. Değer aralığı da n mertebesindeyse işlem sayısı yaklaşık n^2 olur.
  - 2. yöntem: dizinin elemanı başka dizinin indisi olarak kullanılır. Frekans dizisi sıfırlandıktan sonra tek döngü yeter: `B[A[i]]++;`. `if`'e gerek kalmaz. n işlem. Örneğin 1, 8, 3, 6, 9, 3, 0, 8 dizisinde `B[3]` ve `B[8]` 2 olur. Frekans dizisinin boyutu, dizideki en küçük ve en büyük değere göre seçilir (burada 0-9 için 10 eleman).
  - Görüntü (matris) için iki döngüyle bütün pikseller gezilir: `B[A[i][j]] += 1;`. Piksel değerleri 0-255 ise frekans dizisi 256 elemanlıdır.
  - Bir sonraki derste seçim sistemleri örneği (her seçmenin birden fazla adaya oy verdiği durumlar) işlenecektir.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınavlarda sorudaki kısıtlara uyun. "Ben böyle biliyordum" bir gerekçe değildir.
- Pointer'ı anlamak için `y` / `yPtr` / `&` / `*` örneği bu konunun en önemli slaytıdır. Bu basamakları sağlam çıkan pointer konusunda zorlanmaz.
- Bir fonksiyonda değişken değiştirilecekse adresi gönderilir. Aynı fonksiyonun bir parametresi referansla, diğeri değerle aktarılabilir.
- Dizinin adı sabittir, değiştirilemez. Pointer aritmetiği ayrı bir pointer değişkeniyle yapılır.
- Operatör önceliği karışıklığından kaçınmak için parantez kullanın.
- C'de dinamik bellek, `struct`, dosya işlemleri gelecek dönemin konusudur. Bu dönem için gerekli C bilgisi (string'ler hariç) bu derste tamamlanmıştır. Algoritmaları test edebilmek için bu kadarı yeterlidir.
- Pointer operatörlerini, dizilerin ve değişkenlerin aktarımını tekrar edin. Bunlar Sayısal Analiz'de program yazarken de işinize yarayacaktır.

Kısa Tekrar Notları

- `int *p;` pointer tanımı. `p = &x;` adres atama. `*p` gösterilen değer.
- `*&x == x`.
- Referansla çağırma: `void f(int *p)` ve `f(&x)`. Fonksiyonda `*p = ...`.
- Matris parametresi: `int a[][SUTUN]`, satır sayısı ayrı parametre.
- `p + 1` → adres + `sizeof(*p)`.
- `a[i] == *(a + i)`.
- `(*p)++` değeri artırır. `*p++` pointer'ı artırır (ders sonrası nota bakın).

- Sıralı mı: `while (i < n-1 && a[i] <= a[i+1]) i++;` sonra `i == n-1`.
- Frekans: `B[A[i]]++`.

Detaylı Açıklamalar

Derste yazılan pointer örneği

Derste yazılan programın parçaları, ders sonrasında `gcc -Wall` ile derlenip çalıştırıldı:

```
#include <stdio.h>

void swap_value(double x, double y){ double t; t=x; x=y; y=t; }
void swap_ref(double *x, double *y){ double t; t=*x; *x=*y; *y=t; }
void print_array(int *a, int size){
    int i;
    for(i=0;i<size;i++)
        printf("%3d",*(a+i));
    printf("\n");
}
void mutate_array(int *a, int size){ a[3]=444; size--; }

int main(){
    double x=5.0, y=6.0;
    int a[5]={1,2,3};
    int *pa;

    printf("Before: %f -- %f\n",x,y);
    swap_value(x,y);
    printf("After: %f -- %f\n",x,y);       /* 5 -- 6, degismedi */
    swap_ref(&x,&y);
    printf("After-2: %f -- %f\n",x,y);     /* 6 -- 5 */

    print_array(a,5);                      /* 1 2 3 0 0 */
    mutate_array(a,5);
    print_array(a,5);                      /* 1 2 3 444 0 */

    pa=a;
    pa++;                                  /* pa artik a[1]'i gosterir, adres 4 byte artti */
    (*pa)++;                               /* a[1] 2'den 3'e cikti */
    printf("%d %d\n",a[1],*pa);            /* 3 3 */
    return 0;
}
```

Ders sonrasındaki denemede `pa + 1` ile `pa` arasındaki fark 4 byte çıktı. Bu sistemde `sizeof(int)` 4, `sizeof(int *)` 8'dir.

Frekans hesabı

```
int A[8] = {1, 8, 3, 6, 9, 3, 0, 8};
int B[10] = {0};
int i;

for (i = 0; i < 8; i++)
    B[A[i]]++;
```

Ders sonrasında denendi: `B[0]=1`, `B[1]=1`, `B[3]=2`, `B[6]=1`, `B[8]=2`, `B[9]=1`, diğerleri 0.

Ders sonrası notlar

- **Pointer aritmetiğinde artış miktarı:** Derste `pa + 1`'in adresi "`sizeof(int *)` kadar" artırdığı söylendi. Doğrusu, gösterilen tipin boyutu kadar, yani `sizeof(int)` kadar artır-

masıdır. 64 bitlik sistemlerde `sizeof(int *)` 8, `sizeof(int)` 4'tür ve dizi elemanları 4 byte arayla durur. Derste görülen 4 byte'lık artış da `sizeof(int)`'tir.

- **`*pa++` ifadesi:** Derste `*pa++` için "önce adres artırılıyor, sonra o adresteki değer getiriliyor" sonucuna varıldı. Sonek `++` operatörünün önceliği `*`'tan yüksektir, bu yüzden ifade `*(pa++)` olarak okunur. Ama sonek artırma, `pa`'nın eski değerini kullanır: ifadenin değeri, `pa`'nın artırılmadan önce gösterdiği elemandır. İşlem bitince `pa` bir sonraki elemanı gösterir. Ders sonrasında `v = *pa++;` denendi: `pa` `A[0]`'ı (1) gösterirken `v` 1 oldu ve `pa` `A[1]`'e geçti. Değeri artırmak için `(*pa)++` yazılmalıdır.
- **`pa = &A`:** Derste dizi için `pa = A;`, `pa = &A[0];` ve `pa = &A;` yazımlarının hepsinin kullanılabileceği söylendi. İlk ikisi `int *` tipindedir. `&A` aynı adresi verir ama tipi "5 elemanlı int dizisine pointer"dır (`int (*)[5]`). Eski GCC sürümleri bu atamada uyarı verir, yeni sürümler (GCC 14 ve sonrası) derleme hatası verir. `pa = A;` veya `pa = &A[0];` kullanılmalıdır.
- **Sıralılık kontrolünde eşit elemanlar:** Derste kurulan şart `a[i] < a[i+1]`'dir. Bu şartla 1 2 2 3 gibi eşit elemanlı sıralı bir dizi "sıralı değil" olarak görülür. Küçükten büyüğe sıralılık için `a[i] <= a[i+1]` kullanılmalıdır. Ders sonrasında 1 3 3 5 8 9 dizisi bu şartla "sıralı" çıktı.
- **`printf` ve `%lf`:** Derste `double` yazdırırken `%lf` kullanıldı. `printf`'te `float` değerler zaten `double`'a çevrildiği için `%f` yeterlidir. `%lf`, `printf`'te C99'dan beri `%f` ile aynı anlamdadır. C89'da `printf` için tanımlı değildir (`gcc -std=c89 -pedantic` uyarı verir). `scanf`'te ise `double` okumak için `%lf` zorunludur.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.